

PATBMS

Patient Admission, Triage and Bed Management System

Enterprise Systems Analysis, Design and Modelling

Mohammed Yaacoub Abou Chlih

Student ID: 21138726

ENSE706 – Data Process and Software Modelling

Lecturer: Abubakar Siddique

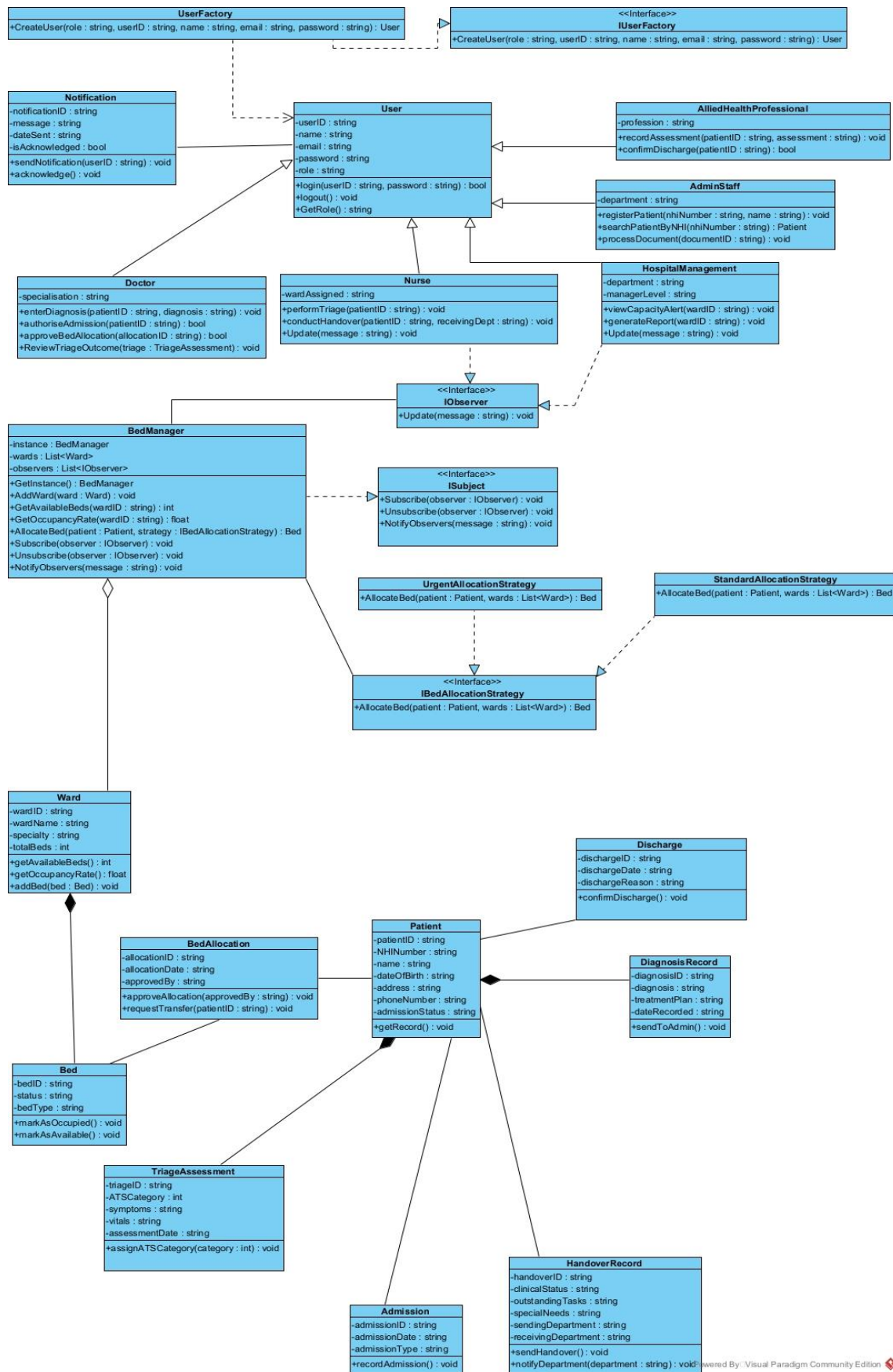
AUT

Due Date: 31 May 2026

Github Repository: <https://github.com/ExplosiveSalad/Data-Process-and-Software-Modelling.git>

Task 5 – Refined Static and Dynamic Modelling

- **Refined UML Class Diagram**



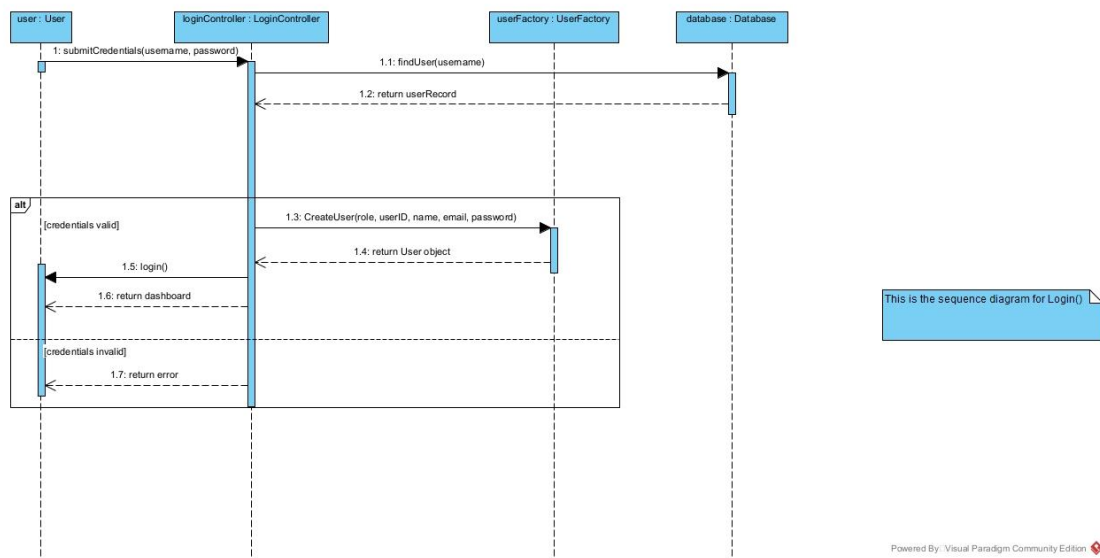
(Figure 3: PATBMS Refined UML Class Diagram (Phase 2))

The refined UML class diagram (Figure 3) extends the Phase 1 design through the introduction of four interfaces and four new classes implementing GoF design patterns (Siddique, 2026). The User hierarchy remains consistent with Phase 1, however Nurse and HospitalManagement now implement the IObserver interface, and the UpdateBedStatus() method has been removed from both Discharge and Nurse, addressing the separation of responsibilities weakness identified in Task 4. Four interfaces were introduced, specifically IObserver, ISubject, IBedAllocationStrategy, and IUserFactory, providing formal abstractions for the Observer, Singleton, Strategy, and Factory patterns. The BedManager class implements both ISubject and the Singleton pattern, centralising all bed and ward management responsibilities. UrgentAllocationStrategy and StandardAllocationStrategy implement IBedAllocationStrategy, providing interchangeable allocation algorithms. UserFactory implements IUserFactory, encapsulating all user object creation. These refinements directly address all four design weaknesses identified in Task 4.

The introduction of interfaces represents a significant structural improvement over Phase 1. IObserver defines the Update() contract that all notification recipients must implement, ensuring BedManager has no direct dependency on Nurse or HospitalManagement. ISubject defines the subscription management contract implemented by BedManager. IBedAllocationStrategy defines the AllocateBed() contract implemented by both strategy classes, allowing allocation logic to be swapped at runtime without modifying BedManager. IUserFactory defines the CreateUser() contract implemented by UserFactory, removing direct subclass instantiation from controllers. The addition of multiplicities on all relationships provides formal specification of cardinality. Ward to Bed is 1 to many, Patient to TriageAssessment is 1 to 1, and Patient to DiagnosisRecord is 1 to many, consistent with standard UML notation (Fowler, 2003).

Sequence Diagram 1 – User Login

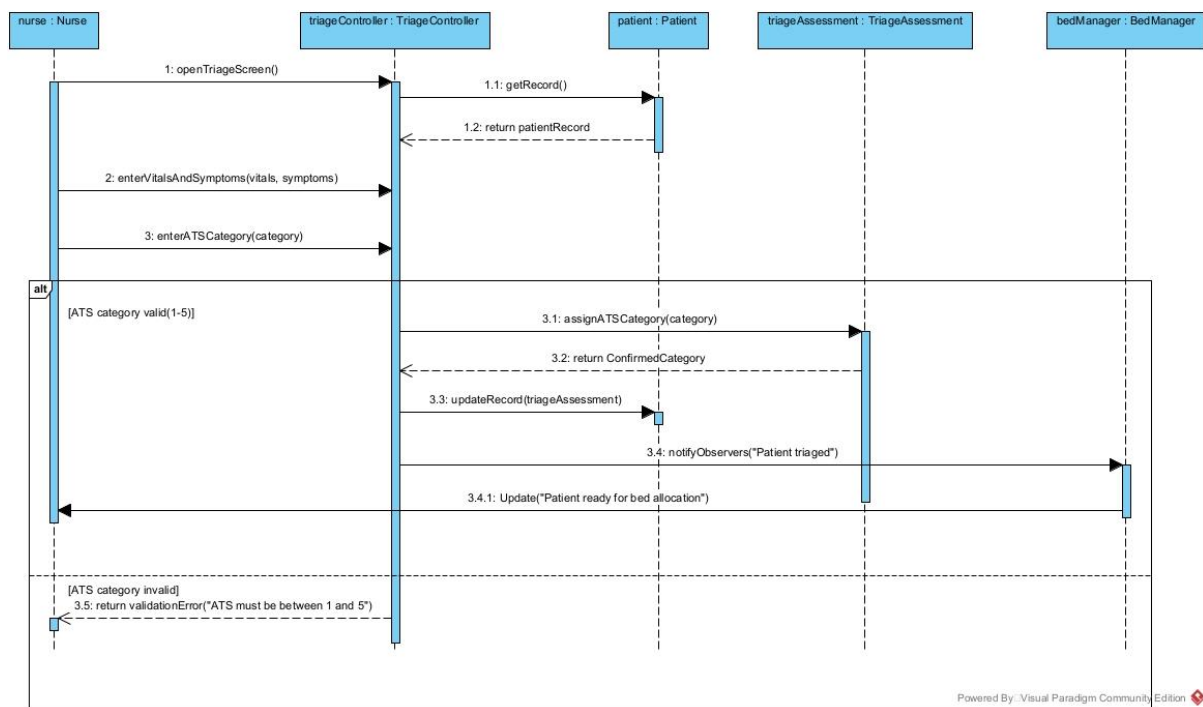
■



(Figure 4: Sequence Diagram – User Login)

The login sequence diagram (Figure 4) models the authentication flow tracing to FR13. Four participants interact including User, LoginController, Database, and UserFactory. The alt fragment captures valid credentials triggering the Factory pattern through CreateUser(), and invalid credentials returning an error message. Activation boxes on LoginController indicate it remains active throughout the entire interaction, coordinating between Database and UserFactory, while the Database activation box is scoped only to the findUser() call, reflecting that the database interaction is a discrete operation. The Factory pattern is demonstrated through the delegation of object creation to UserFactory rather than direct instantiation in the controller, meaning the controller never references Doctor, Nurse, or any specific subclass directly, only the User base type returned by UserFactory (Fowler, 2003).

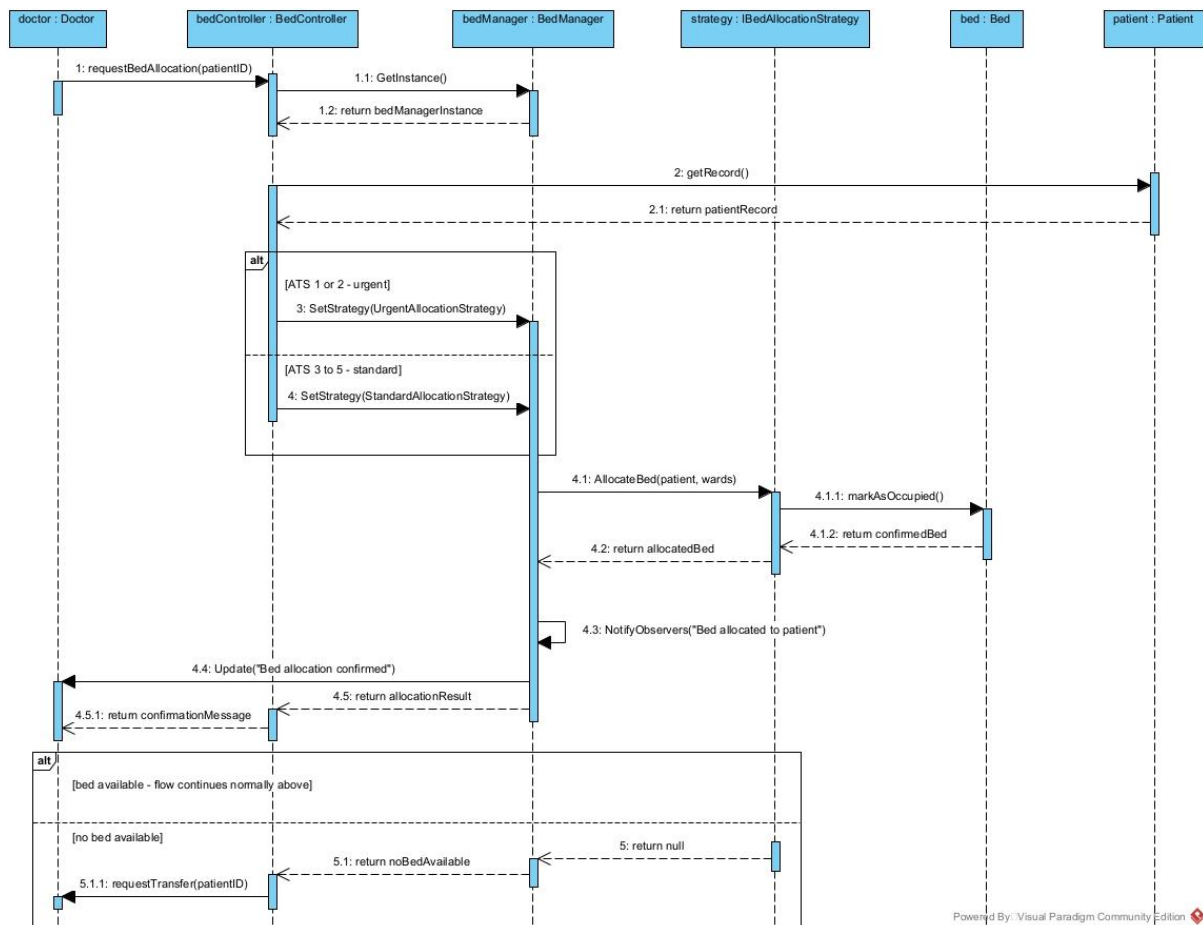
Sequence Diagram 2 – Perform Patient Triage



(Figure 5: Sequence Diagram – Perform Patient Triage)

The triage sequence diagram (Figure 5) models the triage flow tracing to UC1 and FR8. Five participants interact including Nurse, TriageController, Patient, TriageAssessment, and BedManager. The alt fragment captures ATS category validation, with the valid branch proceeding to assignATSCategory() and the invalid branch returning a validation error. The notifyObservers() call on BedManager following successful triage demonstrates the Observer pattern operating within the triage flow. Although triage is primarily a nursing function, the notification reaches all subscribed observers including HospitalManagement, enabling real time capacity awareness across roles. This cross-role notification directly implements FR12 and NFR6 without creating direct dependencies between TriageController and management classes (Gamma et al., 1994). The five-participant interaction in this diagram also demonstrates the separation between responsibilities in the refined design. TriageController acts as the coordinator, delegating triage scoring to TriageAssessment and notification to BedManager, rather than handling all logic internally. This mirrors the class diagram's responsibility allocation where each class has a focused role. The alternative flow for invalid ATS input reflects the business rule implemented in the C# code where values outside 1 to 5 are rejected before a TriageAssessment object is created, preventing invalid data from entering the system (Siddique, 2026).

Sequence Diagram 3 – Allocate Bed

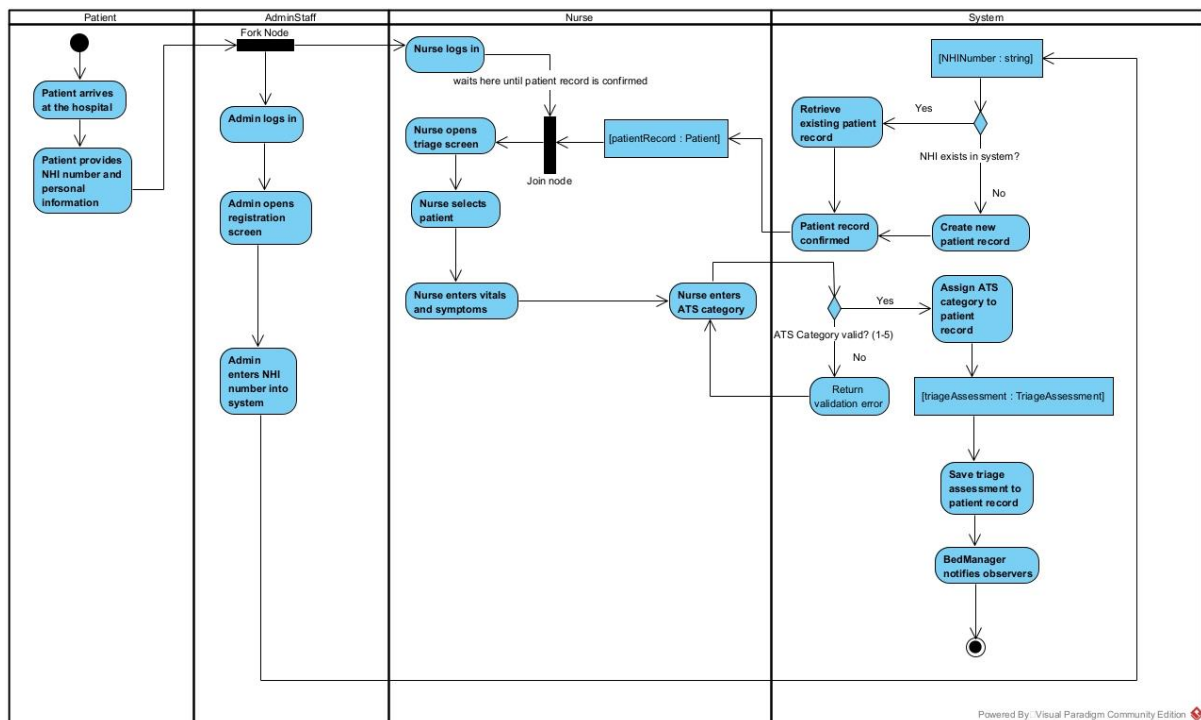


(Figure 6: Sequence Diagram – Allocate Bed with Strategy Pattern)

The bed allocation sequence diagram (Figure 6) models the allocation flow tracing to UC2 and FR9. The Strategy pattern is demonstrated through two alt fragment branches, ATS 1 or 2 triggering UrgentAllocationStrategy and ATS 3 to 5 triggering StandardAllocationStrategy. The selection of the appropriate strategy occurs in BedController based on the patient's ATS category, with ATS 1-2 directing allocation to Titirangi Emergency ward and ATS 3-5 directing to Piha Surgical or Te Henga General Medicine ward. A self-message on BedManager represents the internal NotifyObservers() call, demonstrating the Observer pattern activating automatically following allocation without requiring the controller to trigger notifications directly. A

nested alt fragment captures the no bed available scenario triggering a transfer request (Fowler, 2003).

Activity Diagram – Patient Registration and Triage

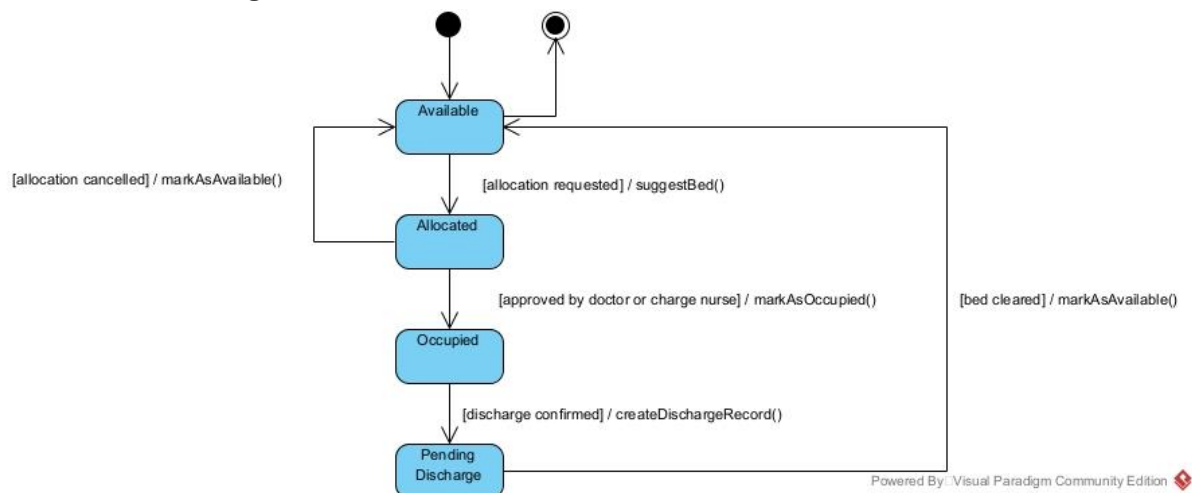


(Figure 7: Activity Diagram - Patient Registration and Triage)

The activity diagram (Figure 7) models the registration and triage workflow across four swim lanes including Patient, AdminStaff, System, and Nurse. Swim lanes enforce clear responsibility boundaries between actors, consistent with the separation of responsibilities principle applied throughout the design. A fork node following patient arrival indicates parallel execution of admin registration and nurse login. Object nodes represent the patientRecord passed from System to Nurse following registration confirmation, and the triageAssessment passed to the save operation following ATS assignment. The object nodes reflect the composition relationships modelled in the class diagram, with patientRecord as a Patient object and triageAssessment as a TriageAssessment object, demonstrating alignment between static and dynamic models. A decision node with loop back captures the ATS validation business rule, tracing to FR8 (Fowler, 2003). The fork and join nodes in the activity diagram also reflect a real clinical workflow consideration. Nurses must be logged in and ready before a

patient record is confirmed, ensuring the triage screen is accessible without delay. The join node enforces that both the patient record and the nurse login must be complete before triage can begin, preventing a scenario where a nurse attempts to triage a patient whose record has not yet been confirmed by administrative staff. This workflow constraint is enforced in the C# implementation through the patient null check in TriageController.cs (Siddique, 2026).

State Machine Diagram – Bed Status



(Figure 8: State Machine Diagram – Bed Status)

The state machine diagram (Figure 8) models the lifecycle of a Bed object across four states including Available, Allocated, Occupied, and PendingDischarge. Transitions are labelled with guard conditions and actions. The Available to Allocated transition implements FR9. The Occupied to PendingDischarge transition implements FR11. The Allocated to Available transition captures the allocation rejection scenario from UC2 Alternative Flow 2. The PendingDischarge state was introduced to represent the clinical reality that discharge is a multi-step process requiring both clinical confirmation and bed clearance before the bed becomes available, preventing premature bed reallocation and supporting the data integrity of the bed dashboard (FR4). The state machine is directly implemented in the BedManager Singleton through the CheckOccupancyThreshold() method and bed status update logic, demonstrating alignment between the UML model and C# implementation.

Task 6: Design Pattern Application and Implementation

Four GoF design patterns were applied to address the weaknesses identified in Task 4 (Gamma et al., 1994). Two patterns are Creational, Factory and Singleton, addressing object creation and instance management respectively. Two patterns are Behavioural,

Observer and Strategy, addressing communication and algorithm flexibility respectively. Each pattern is represented in the refined UML class diagram (Figure 3) and implemented in the C# web application codebase. The following sections describe each pattern's problem context, UML structure, C# implementation, and impact on the system design.

Factory Pattern (Creational – GoF)

Problem Addressed:

The Phase 1 design required controllers to directly instantiate specific User subclasses, creating tight coupling between the controller layer and the model layer. This violated the principle of low coupling identified as a weakness in Task 4.

UML Structure:

As shown in Figure 3, UserFactory implements IUserFactory and holds a dependency relationship to User. The factory encapsulates the decision of which subclass to instantiate, removing this responsibility from the controller. (Gamma et al., 1994). The dependency relationship from UserFactory to User in Figure 3 indicates that UserFactory temporarily uses User objects during creation but does not own or store them. This is the weakest form of relationship in UML, appropriate here because the factory's sole responsibility is instantiation rather than management (Fowler, 2003). The five concrete subclasses Doctor, Nurse, AdminStaff, AlliedHealthProfessional, and HospitalManagement all inherit from User, meaning UserFactory only needs to reference the parent type in its return signature.

C# Implementation:

UserFactory.cs implements a switch statement that returns Doctor, Nurse, AdminStaff, AlliedHealthProfessional, or HospitalManagement objects based on a role string parameter. An ArgumentException is thrown for invalid roles, enforcing the business rule that only valid roles can be created. Controllers call UserFactory.CreateUser() rather than instantiating subclasses directly. In the ASP.NET Core web application, the Factory pattern is applied in AccountController.cs where user login retrieves a user record from the database and the appropriate subclass behaviour is available through the User hierarchy. The pattern ensures that regardless of which role logs in, the controller interacts with a User reference rather than a specific subclass, maintaining loose coupling throughout the controller layer.

Impact:

Adding a new user role requires only a new subclass of User and a new case in UserFactory, with no changes to any controller, directly supporting NFR5 (Gamma et al., 1994).

Singleton Pattern (Creational – GoF)**Problem Addressed:**

The Phase 1 design had no centralised bed management mechanism, meaning bed state could be managed inconsistently across multiple objects.

UML Structure:

As shown in Figure 3, BedManager implements ISubject and contains a private static instance field, a private constructor, and a public static GetInstance() method (Gamma et al., 1994).

C# Implementation:

BedManager.cs enforces the Singleton through three mechanisms: a private static BedManager instance field, a private constructor initialising the wards and observers lists, and a GetInstance() method that creates the instance on first call and returns it on all subsequent calls. In the ASP.NET Core web application, BedManager is accessed via BedManager.GetInstance() in BedController, TriageController, DischargeController, and indirectly through notification calls in HandoverController. Each controller accesses the same instance, ensuring bed state changes made in one controller are immediately visible to all others.

Impact:

A single consistent BedManager instance manages all bed state throughout the application lifecycle, directly supporting FR4 and FR9 (Gamma et al., 1994).

Observer Pattern (Behavioural – GoF)

Problem Addressed:

The Phase 1 design had no automatic notification mechanism, failing to implement FR12 and NFR6 which require capacity alerts to be sent automatically to relevant personnel.

UML Structure:

As shown in Figure 3, BedManager implements ISubject and maintains a list of IObserver objects. Nurse and HospitalManagement implement IObserver through the Update() method (Gamma et al., 1994). The association relationship from BedManager to IObserver in Figure 3 represents the observer list maintained internally by BedManager. This is a one-to-many association, with one BedManager holding references to multiple IObserver objects. The realisation relationships from Nurse and HospitalManagement to IObserver indicate these classes formally implement the Update() contract. This structure means that adding AlliedHealthProfessional as a future observer would require only adding the IObserver realisation to that class and calling Subscribe() at startup (Fowler, 2003).

C# Implementation:

BedManager.cs contains Subscribe(), Unsubscribe(), and NotifyObservers() methods. The CheckOccupancyThreshold() method automatically triggers NotifyObservers() when occupancy reaches 60% or 80%, implementing the automatic alert behaviour required by NFR6. In the ASP.NET Core web application, notifications are persisted to the SQLite database through NotificationController.CreateNotification() which is called after every significant system event including triage, bed allocation, discharge, and handover. This ensures the Notifications page displays a complete audit trail of system events accessible to all authorised users, directly implementing FR12.

Impact:

BedManager has no knowledge of specific observer classes, only IObserver, achieving low coupling. New notification recipients require only implementing IObserver and subscribing to BedManager. (Gamma et al., 1994)

Strategy Pattern (Behavioural – GoF)

Problem Addressed:

The Phase 1 design had no flexible bed allocation mechanism, failing to implement the clinical priority rules of FR9 which requires allocation based on severity and ward specialty.

UML Structure:

As shown in Figure 3, UrgentAllocationStrategy and StandardAllocationStrategy implement IBedAllocationStrategy. BedManager holds an association to the interface (Gamma et al., 1994).

C# Implementation:

BedController.cs implements the strategy selection logic directly, querying the database for available beds filtered by WardID and BedType based on the patient's ATS category. ATS 1-2 patients are directed to Titirangi Emergency ward prioritising High Dependency beds, ATS 3-4 patients to Piha Surgical ward, and ATS 5 patients to Te Henga General Medicine ward.

Impact:

Adding a new allocation strategy requires only a new class implementing IBedAllocationStrategy with no changes to BedManager or BedController, consistent with the Open/Closed principle (Gamma et al., 1994).

The combined application of these four patterns transforms the Phase 1 design from a tightly coupled collection of classes into a loosely coupled, extensible architecture. The Factory and Singleton patterns address structural concerns at the object creation level, while Observer and Strategy address behavioural concerns at the runtime interaction level. Together they directly resolve all four design weaknesses identified in Task 4, while introducing a web-based application that demonstrates the system functioning as a genuine clinical management tool (Gamma et al., 1994).

Task 7 – Comparative Design Analysis

This section provides a critical comparison of the PATBMS design before and after the application of GoF design patterns. The analysis addresses changes to class structure, improvements in cohesion and coupling, impact on extensibility and maintainability, and consequences for C# implementation complexity. All comparisons reference the Phase 1 class diagram (Figure 2) and the Phase 2 refined class diagram (Figure 3) as primary evidence (Siddique, 2026).

Changes to Class Structure and Responsibility Allocation

The Phase 1 design contained 16 classes with no formal interfaces and responsibility violations including UpdateBedStatus() in both Discharge and Nurse. The refined design contains 20 classes and 4 interfaces. UpdateBedStatus() is consolidated within BedManager as the single authority for bed state changes. IObserver, ISubject, IBedAllocationStrategy, and IUserFactory provide formal contracts defining system behaviours independently of their implementations (Figure 3). In Phase 1, Discharge contained UpdateBedStatus() creating a responsibility overlap with Bed. In Phase 2 this method is removed from Discharge entirely and consolidated within BedManager, which now serves as the authoritative source for all bed state transitions. Similarly, Nurse no longer contains UpdateBedStatus(), as this responsibility has been properly assigned to BedManager. The result is that each class in the refined design has a clearly bounded set of responsibilities, consistent with the Single Responsibility Principle.

Improvements in Cohesion and Reductions in Coupling

The Phase 1 design exhibited low cohesion in Discharge and high coupling between Program.cs and every User subclass. Following pattern application, each class has a focused responsibility. BedManager manages bed state exclusively. Nurse and HospitalManagement receive notifications through IObserver without direct BedManager dependency. BedController implements allocation logic using WardID filtering rather than referencing concrete strategy classes directly, with the strategy selection logic determining which ward is targeted based on ATS category. In Phase 1, Program.cs contained direct instantiation of Doctor, Nurse, AdminStaff, AlliedHealthProfessional, and HospitalManagement objects, creating five direct dependencies between the entry point and specific model classes. In Phase 2, the ASP.NET Core controllers interact with User references created through UserFactory, reducing these dependencies to a single factory reference. This measurably reduces

affluent coupling in the controller layer and improves the testability of the system (Siddique, 2026).

Impact on Extensibility, Maintainability, and Accommodation of New Requirements

Prior to pattern application, adding a new user role required modifications across multiple files. Following pattern application, a new role requires only a new User subclass and a UserFactory case. A new notification recipient requires only IObserver implementation. A new allocation strategy requires only a new IBedAllocationStrategy class. These improvements directly address NFR5 and demonstrate the Open/Closed principle in practice. The practical impact is demonstrated in the web application where the notification system was extended to cover five different system events, triage, bed allocation, discharge, handover, and capacity threshold, without modifying BedManager's core structure. Each new notification type was added by calling NotifyObservers() with a new message string. The before and after comparison in Table 1 quantifies these improvements concisely. The addition of four interfaces and four new classes represents a 25% increase in structural components, which is a modest cost relative to the extensibility and responsibility clarity gained. The shift from a console application with no database to a fully functional ASP.NET Core web application with SQLite persistence demonstrates that the refined design is not merely theoretical but has been validated through a working implementation that exercises all four patterns in realistic clinical workflows (Siddique, 2026).

Consequences for C# Implementation Complexity

Pattern application introduced additional complexity that must be understood and managed throughout development. The Singleton pattern requires careful static instance management outside ASP.NET Core's dependency injection lifecycle. Because BedManager is accessed via a static GetInstance() call rather than through constructor injection, it cannot easily be replaced with a mock object during unit testing, reducing testability. The Observer pattern requires explicit subscription management at application startup, meaning if a new observer is added to the codebase but not subscribed in Program.cs, it will never receive notifications. The Strategy pattern introduces indirection requiring the controller to select a strategy before calling AllocateBed(), meaning the strategy selection logic is spread across BedController rather than encapsulated within BedManager. The Factory pattern adds a layer of indirection that requires developers to navigate UserFactory before understanding how specific user types are created. These complexities are justified by the extensibility and

maintainability gains provided, but represent a genuine increase in cognitive load for developers new to the codebase (Siddique, 2026).

Table 1: Before and After Pattern Application

Aspect	Phase 1	Phase 2
Total Classes	16	20
Interfaces	0	4
UpdateBedStatus location	Discharge and Nurse	BedManager only
User Creation	Direct instantiation	UserFactory
Notification mechanism	Manual print statements	Observer pattern
BedAllocation logic	None	Strategy pattern
Access control	Session checks only	Session checks with role-based navigation and factory-created user objects
Database	None	SQLite via Entity Framework
User interface	Console application	ASP.NET Core web application

Task 8: Critical Reflection

This section reflects critically on the design decisions made throughout Phase 2, examining trade-offs introduced by pattern application, remaining limitations, lessons learned from aligning UML with C# implementation, and planned future improvements. The reflection draws on both the theoretical foundations of GoF patterns (Gamma et al., 1994) and the practical experience of implementing a fully functional ASP.NET Core web application with a SQLite database.

Design Trade-offs Introduced by Design Patterns

Each pattern introduced trade-offs between structural quality and implementation simplicity that required deliberate evaluation (Gamma et al., 1994). The Singleton pattern ensures consistent bed state management across all controllers but introduces global mutable state that complicates unit testing, as the same instance persists across test cases. The Factory pattern removes direct coupling between controllers and model subclasses but adds an additional file and indirection layer that increases the number of components a developer must understand. The Observer pattern enables automatic notifications without direct dependencies but requires careful subscription management at startup and increases the complexity of application initialisation. The Strategy pattern enables runtime algorithm selection but moves decision logic into the

controller rather than encapsulating it within BedManager, representing a partial compromise of the pattern's intended encapsulation benefit. These trade-offs were accepted because the extensibility and maintainability benefits outweigh the added complexity for a system expected to serve a growing and diverse West Auckland population over multiple development phases.

Limitations of the Final Design

The current design retains several limitations. Authentication uses plain text passwords, violating NFR3's encryption requirement. Role-based access control is enforced through session checks but not structurally within the class hierarchy. BedManager operates outside ASP.NET Core's dependency injection system. The patient-bed relationship relies on a string BedID field rather than a formal foreign key, which may create data integrity issues at scale. These limitations represent planned improvements rather than design oversights. The decision to use plain text passwords was made to maintain focus on structural design patterns rather than authentication security during Phase 2. The SQLite database was selected as a development alternative due to network constraints preventing direct connection to the university MySQL server from off-campus. Both limitations are documented as known issues with clear resolution paths, consistent with professional software engineering practice of acknowledging technical debt explicitly.

Lessons Learned from Aligning UML with C#

Aligning UML models with C# implementation revealed several practical insights. Composition relationships required additional consideration in Entity Framework as object lifecycles differ from in-memory graphs. The Singleton assumed simple static instantiation but ASP.NET Core's request-scoped lifecycle adds complexity. The Strategy pattern required selection logic in the controller rather than BedManager, representing a pragmatic deviation from ideal pattern structure. UML models represent ideal structural relationships that may require adaptation when implemented within a specific framework. The BedManager Singleton, for example, is modelled in UML as a straightforward static instance but in ASP.NET Core would ideally be registered as a singleton service within the dependency injection container. This gap between ideal design and practical implementation highlights the importance of understanding framework constraints before finalising UML models (Fowler, 2003). Additionally, Entity Framework's approach to object lifecycle management required the introduction of parameterless constructors on all model classes, a requirement not reflected in the UML class diagram, demonstrating that implementation concerns can necessitate deviations from the modelled design. A further lesson learned concerns the relationship between UML diagrams and the report writing process. Producing written descriptions of each diagram forced a deeper evaluation of design decisions that might otherwise have been left implicit. For example, articulating why the PendingDischarge state was

introduced in the state machine diagram required revisiting the clinical workflow and confirming that the implementation correctly reflected the modelled behaviour. This iterative process of modelling, implementing, and then describing the design improved both the quality of the documentation and the accuracy of the implementation, reinforcing the value of UML as a communication and verification tool (Fowler, 2003).

Future Improvements

In retrospect, the database schema would be designed with formal foreign key relationships from the beginning. Authentication would implement BCrypt password hashing. BedManager would be registered within ASP.NET Core's dependency injection container. The role attribute would be implemented as an enumeration. These reflections inform planned improvements including a proper audit logging mechanism to satisfy NFR3. The development process also revealed that alignment between UML models and C# implementation would benefit from iterative refinement. Future phases would adopt a more incremental approach, modelling a small number of classes, implementing them, then refining the model before proceeding, rather than completing the full UML design before any implementation begins. This approach would reduce the divergence between modelled and implemented designs observed particularly in the Entity Framework integration and Singleton lifecycle management. The Phase 2 implementation also demonstrated the value of the web-based approach over the Phase 1 console application. The ASP.NET Core MVC architecture provided a clear separation between controller logic, model classes, and view rendering, which naturally reinforced the design pattern boundaries established in the UML models. Future development would extend this architecture by introducing a dedicated service layer between controllers and models, further reducing controller complexity and aligning more closely with the ideal pattern structures modelled in the class diagram. The introduction of unit testing using a framework such as xUnit would also be prioritised, requiring the BedManager Singleton to be refactored into a dependency-injected service to support test isolation. These improvements would bring the implementation into closer alignment with professional ASP.NET Core development practices and the structural quality objectives of the GoF pattern application (Gamma et al., 1994).

References

Health New Zealand. (2024). Waitakere Hospital.

<https://www.health.govt.nz/regulation-legislation/certification-of-health-care-services/certified-providers/public-hospitals/waitakere-hospital>

Fowler, M. (2003). UML distilled: A brief guide to the standard object modeling language (3rd ed.). Addison-Wesley.

Healthpoint. (2024). Waitakere Hospital. <https://www.healthpoint.co.nz/waitakere-hospital/>

Siddique, A. (2026). Class Diagrams and Static Modelling [PowerPoint slides]. ENSE706 Data Process and Software Modelling, Auckland University of Technology.

Siddique, A. (2026). Requirement Analysis Using UML [PowerPoint slides]. ENSE706 Data Process and Software Modelling, Auckland University of Technology.

Siddique, A. (2026). Use Case Diagrams and Use Case Modelling [PowerPoint slides]. ENSE706 Data Process and Software Modelling, Auckland University of Technology.

Siddique, A. (2026). Sequence Diagrams and Dynamic Modelling [PowerPoint slides]. ENSE706 Data Process and Software Modelling, Auckland University of Technology.

Siddique, A. (2026). Activity Diagrams and State Machine Modelling [PowerPoint slides]. ENSE706 Data Process and Software Modelling, Auckland University of Technology.

Siddique, A. (2026). Design Patterns, Design Patterns-ii, Design Patterns-iii, Design Patterns-iv [PowerPoint slides]. ENSE706 Data Process and Software Modelling, Auckland University of Technology.

